

Name: vaishnavi rajendra gayake

Subject:AIML

College name:T.b.girwalkar polytechnic college ambajogai

Branch:electronics and computer

Q1. Find a dataset that has at least one continuous target variable (e.g., Price, Salary, Sales, Temperature, House Price, Car Mileage, etc.).

Q2. (Dataset Selection & Initial Exploration)

Load the dataset and show first 10 rows.

Write 5-6 lines explaining why you chose this dataset and what continuous value you want to predict

```
import pandas as pd
```

```
df = pd.read_csv("Real estate.csv")
```

```
print("First 10 Rows of the Dataset:")
```

```
print(df.head(10))
```

Q3. (Missing & Duplicate Values Analysis)

- Check for missing values in each column and calculate their percentage.
- Handle missing values appropriately (drop or fill). Check and remove duplicate rows if any.
- Write your observations and steps taken.

```
missing_values = df.isnull().sum()
```

```
missing_percentage = (missing_values / len(df)) * 100
```

```
missing_summary = pd.DataFrame({
```

```
    'Missing Values': missing_values,
```

```
    'Percentage': missing_percentage
```

```
})
```

```
print("Missing Values Summary:")
```

```

print(missing_summary)

df = df.dropna()

print("Shape after handling missing values:", df.shape)

duplicate_rows = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_rows)

df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)

print("\nMissing Values After Cleaning:")

print(df.isnull().sum())

print("\nDuplicate Rows After Cleaning:")

print(df.duplicated().sum())

```

#### Q4.(Statistical Summary)

e Use describe() to generate statistical summary of numeric columns • Identify min, max, mean, and median of the target variable. • Write at least 4 meaningful observations about the data distribution,

```

print("Statistical Summary of Numeric Columns:")

print(df.describe())

target = 'Y house price of unit area'

print("Minimum House Price :", df[target].min())

print("Maximum House Price :", df[target].max())

print("Mean House Price  :", df[target].mean())

print("Median House Price :", df[target].median())

```

Q5.(Histogram Analysis of Numeric Features,

- Plot histograms for all important numeric columns (including the target variable). . Write detailed insights for each plot.

```
import matplotlib.pyplot as plt
```

```
numeric_cols = [
```

```
    'X2 house age',
```

```
    'X3 distance to the nearest MRT station',
```

```
    'X4 number of convenience stores',
```

```
    'Y house price of unit area'
```

```
]
```

```
df[numeric_cols].hist(figsize=(12, 8), bins=20)
```

```
plt.suptitle("Histograms of Numeric Features")
```

```
plt.show()
```

1. House Age (X2 house age):

The histogram of house age shows that properties of different ages are available in the dataset. Most houses are concentrated in the low to medium age range, while very old houses are fewer in number. This indicates that newer and moderately aged houses are more common in the market.

2. Distance to Nearest MRT Station (X3 distance to the nearest MRT station):

The distribution is positively skewed, meaning that most houses are located relatively close to MRT stations, while only a small number of houses are located very far away. This suggests that accessibility to public transportation is an important factor in property locations.

3. Number of Convenience Stores (X4 number of convenience stores):

The histogram indicates that most residential areas have a moderate number of convenience stores nearby. Very few locations have either no stores or a very large number of stores. This reflects balanced urban development around residential properties.

#### 4. House Price of Unit Area (Y house price of unit area):

The target variable shows a slightly right-skewed distribution. Most house prices are concentrated in the medium price range, while a few properties have exceptionally high prices. These high-priced properties may act as outliers and can influence regression models.

#### 5. Overall Observation:

The numeric features exhibit different distributions and scales, indicating that feature scaling or normalization may improve model performance. The presence of skewness and outliers should be considered during model building and evaluation.

#### Q6.(Count Plots of Categorical Features)

e Identify categorical columns in your dataset. . Create count plots for all categorical features using seaborn. • Analyze the distribution and write business/domain-related insights.

```
categorical_cols = df.select_dtypes(include=['object', 'category']).columns
```

```
print("Categorical Columns:")
```

```
print(list(categorical_cols))
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
for col in categorical_cols:
```

```
    plt.figure(figsize=(6,4))
```

```
    sns.countplot(x=col, data=df)
```

```
    plt.title(f'Count Plot of {col}')
```

```
    plt.xticks(rotation=45)
```

```
    plt.show()
```

```

Q7.import seaborn as sns

import matplotlib.pyplot as plt

corr_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(10,8))

sns.heatmap(

    corr_matrix,

    annot=True,

    cmap='coolwarm',

    fmt='.2f',

    linewidths=0.5

)

plt.title('Correlation Heatmap of Numeric Features')

plt.show()(Correlation Heatmap)

```

• Create a correlation heatmap for all numeric features. • Highlight which features are strongly correlated with the target variable • Write your observations regarding feature relationships.

```

target_corr =

corr_matrix['Y house price of unit area'].sort_values(ascending=False)

print("Correlation with Target Variable:")

print(target_corr)

```

Q8.(Feature Identification) Clearly define:

• Independent Features (X) - input features • Dependent Feature (y) – continuous target variable you want to predict Justify your selection with reasoning.

```
X = df[['X1 transaction date',  
        'X2 house age',  
        'X3 distance to the nearest MRT station',  
        'X4 number of convenience stores',  
        'X5 latitude',  
        'X6 longitude']]
```

```
y = df['Y house price of unit area']
```

```
print("Shape of X:", X.shape)
```

```
print("Shape of y:", y.shape)
```

The independent features (X) were selected because they represent factors that can influence the value of a property. Features such as house age, distance to the nearest MRT station, and number of convenience stores directly affect the attractiveness and accessibility of a location. Latitude and longitude capture geographical information, which is an important factor in real estate valuation. The transaction date may reflect changes in market conditions over time.

The dependent feature (y) is the house price of unit area because it is the outcome variable that depends on the characteristics of the property and its surroundings. Since house price is a continuous numerical value, this problem is a regression problem and is suitable for algorithms such as Linear Regression, Decision Tree Regression, and Random Forest Regression.

Q9.(Encoding Categorical Variables) Convert all categorical columns to numeric form using suitable encoding techniques (One-Hot Encoding),

Show before and after transformation for at least two columns.

Explain your encoding choices.

```
categorical_cols = df.select_dtypes(include=['object', 'category']).columns  
  
print("Categorical Columns:")
```

```
print(list(categorical_cols))

df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("Shape before encoding:", df.shape)

print("Shape after encoding :", df_encoded.shape)
```

The selected Real Estate Valuation Dataset does not contain any categorical variables; therefore, no encoding was necessary. All input features such as house age, distance to the nearest MRT station, latitude, longitude, and house price are already in numerical form and can be directly used for machine learning models.

If categorical variables were present, One-Hot Encoding would be the preferred technique because it converts categorical values into binary numeric columns without introducing any artificial ordering among categories. This method is widely used in regression and classification problems because machine learning algorithms require numerical input values.

Since no categorical features exist in this dataset, the original dataset and the encoded dataset remain the same.

Q10.(Feature Scaling) Apply Standard Scaling on the independent features using StandardScaler.

- Show the first 5 rows of the scaled features.

```
from sklearn.preprocessing import StandardScaler

import pandas as pd

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

print("First 5 Rows of Scaled Features:")

print(X_scaled_df.head())
```

Standard Scaling was applied to all independent features using the StandardScaler class from scikit-learn. This method transforms each feature so that it has a mean of 0 and a standard deviation of 1. Feature scaling is important because variables in the dataset have different

ranges and units. For example, the distance to the MRT station is measured in meters and has much larger values than latitude or longitude. Without scaling, features with larger magnitudes may dominate the learning process of machine learning algorithms. Standard Scaling improves the performance of models such as Linear Regression, Support Vector Regression, and K-Nearest Neighbors.